Sri Lanka Institute of Information Technology
Faculty of Computing — Department of Information Technology
IT2140 : Database Design and Development — Year 2, Semester 1 (2026)

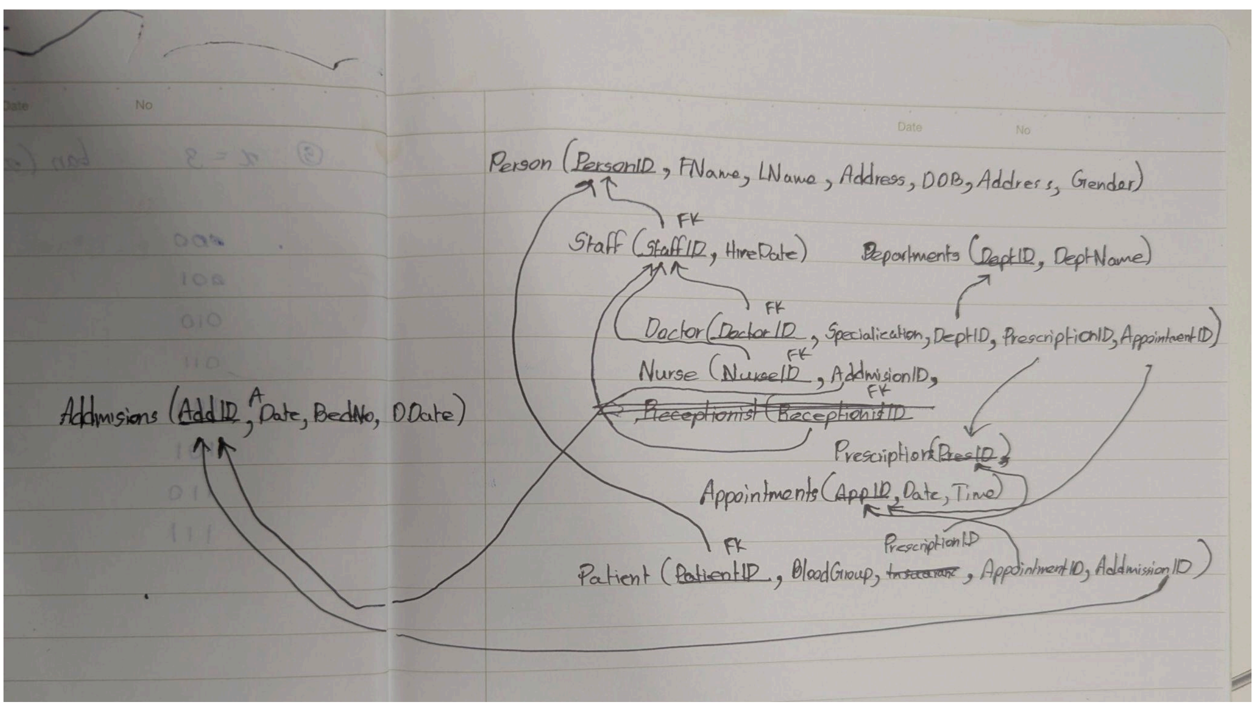
Requirement Specification Document

Web-Based Hospital Management System (Part 2 continuation)

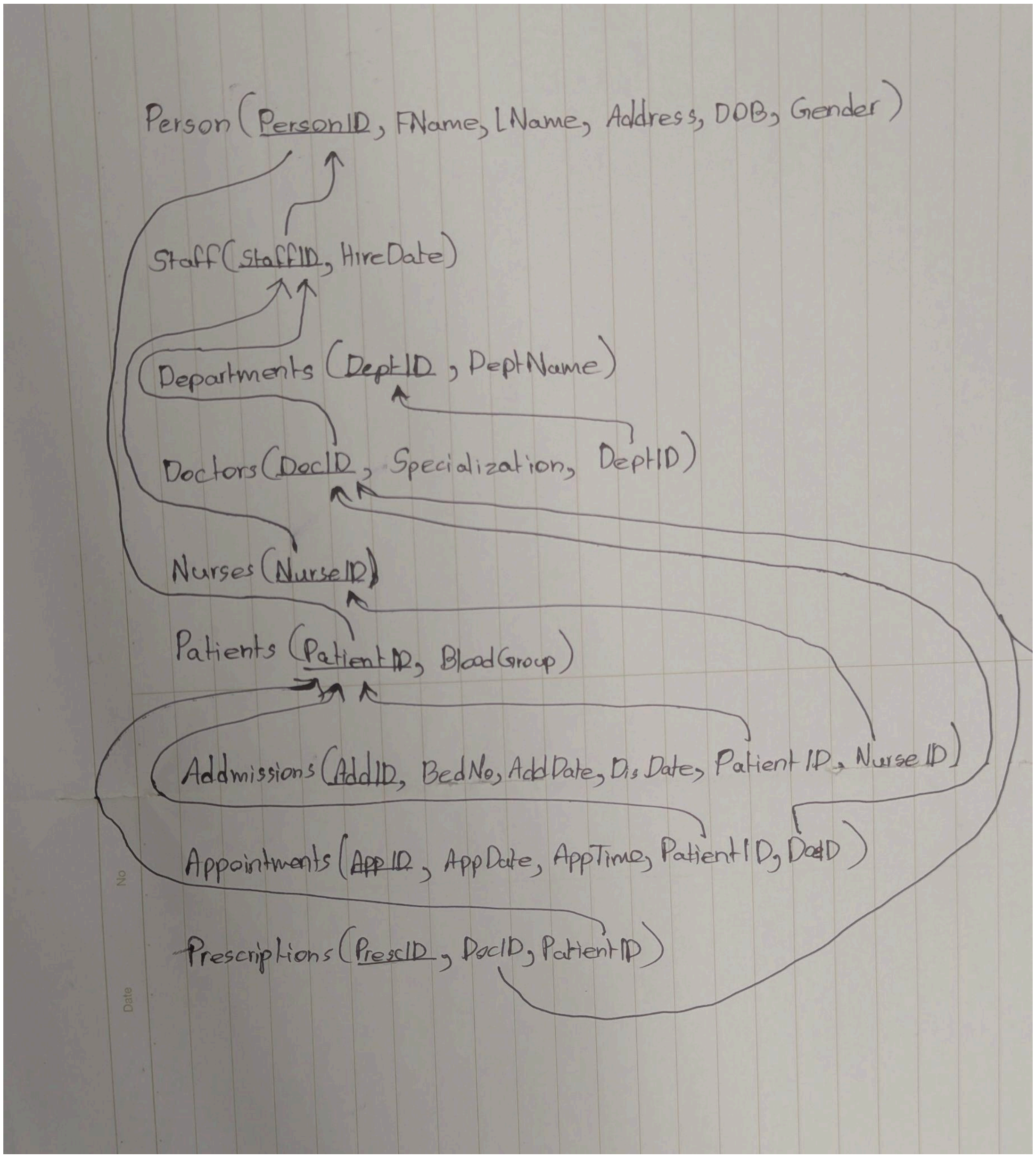
Group ID	DDD_Jan_D_01
Group Members	IT26100409 — Senadheera S.M.K | IT26100329 — Gooneekara N.J
System Topic	Web-Based Hospital Management System
Client (context)	Riverside General Hospital — a mid-sized 250-bed public hospital

Part A: Relational Schema

The first attempt at creating the schema



Finalized schema with less redundancies



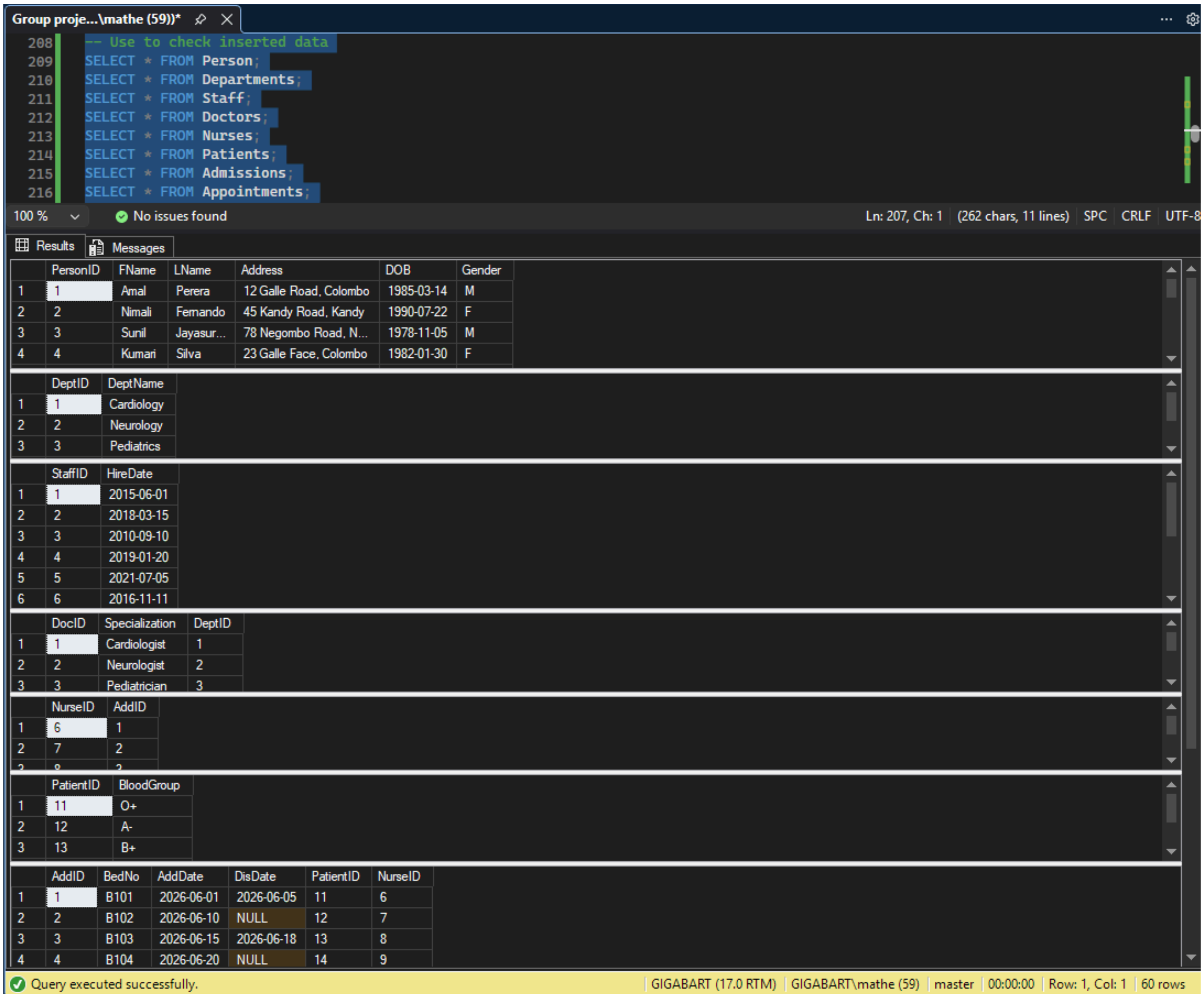
Part B: SQL DDL Implementation

```sql
-- Part B: Initialization of tables aka DDL

CREATE TABLE Person
(
    PersonID     INT IDENTITY(1,1) NOT NULL,
    FName        VARCHAR(50)       NOT NULL,
    LName        VARCHAR(50)       NOT NULL,
    Address      VARCHAR(255)      NULL,
    DOB          DATE              NULL,
```

```sql
    Gender      CHAR(1)           NULL,
    CONSTRAINT PK_Person PRIMARY KEY (PersonID)
);

CREATE TABLE Staff
(
    StaffID     INT          NOT NULL,
    HireDate    DATE         NULL,
    CONSTRAINT PK_Staff PRIMARY KEY (StaffID),
    CONSTRAINT FK_Staff_Person FOREIGN KEY (StaffID)
        REFERENCES Person (PersonID)
        ON DELETE CASCADE -- Deletes the row if parent (Person Table) row is deleted
);

CREATE TABLE Departments
(
    DeptID      INT IDENTITY(1,1) NOT NULL,
    DeptName    VARCHAR(100)      NOT NULL,
    CONSTRAINT PK_Departments PRIMARY KEY (DeptID)
);


CREATE TABLE Doctors
(
    DocID           INT          NOT NULL,
    Specialization  VARCHAR(100) NULL,
    DeptID          INT          NULL,
    CONSTRAINT PK_Doctors PRIMARY KEY (DocID),
    CONSTRAINT FK_Doctors_Staff FOREIGN KEY (DocID)
        REFERENCES Staff (StaffID)
        ON DELETE CASCADE, -- Deletes the row if parent (Staff Table) row is deleted
    CONSTRAINT FK_Doctors_Departments FOREIGN KEY (DeptID)
        REFERENCES Departments (DeptID)
);


CREATE TABLE Nurses
(
    NurseID     INT NOT NULL,
    AddID       INT NULL,
    CONSTRAINT PK_Nurses PRIMARY KEY (NurseID),
    CONSTRAINT FK_Nurses_Staff FOREIGN KEY (NurseID)
        REFERENCES Staff (StaffID)
        ON DELETE CASCADE, -- Deletes the row if parent (Staff Table) row is deleted
);

CREATE TABLE Patients
(
    PatientID   INT          NOT NULL,
    BloodGroup  VARCHAR(5)   NULL,
    CONSTRAINT PK_Patients PRIMARY KEY (PatientID),
    CONSTRAINT FK_Patients_Person FOREIGN KEY (PatientID)
        REFERENCES Person (PersonID)
        ON DELETE CASCADE -- Deletes the row if parent (Person Table) row is deleted
);

CREATE TABLE Admissions
(
    AddID       INT IDENTITY(1,1) NOT NULL,
    BedNo       VARCHAR(10)       NULL,
    AddDate     DATE              NULL,
    DisDate     DATE              NULL,
    PatientID   INT               NULL,
    NurseID     INT               NULL,
```

```sql
    CONSTRAINT PK_Admissions PRIMARY KEY (AddID),
    CONSTRAINT FK_Admissions_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients (PatientID),
    CONSTRAINT FK_Admissions_Nurses FOREIGN KEY (NurseID)
        REFERENCES Nurses (NurseID)
);

CREATE TABLE Appointments
(
    AppID       INT IDENTITY(1,1) NOT NULL,
    AppDate     DATE              NULL,
    AppTime     TIME              NULL,
    PatientID   INT               NULL,
    DocID       INT               NULL,
    CONSTRAINT PK_Appointments PRIMARY KEY (AppID),
    CONSTRAINT FK_Appointments_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients (PatientID),
    CONSTRAINT FK_Appointments_Doctors FOREIGN KEY (DocID)
        REFERENCES Doctors (DocID)
);

CREATE TABLE Prescriptions
(
    PresID      INT IDENTITY(1,1) NOT NULL,
    DocID       INT               NULL,
    PatientID   INT               NULL,
    CONSTRAINT PK_Prescriptions PRIMARY KEY (PresID),
    CONSTRAINT FK_Prescriptions_Doctors FOREIGN KEY (DocID)
        REFERENCES Doctors (DocID),
    CONSTRAINT FK_Prescriptions_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients (PatientID)
);
```

Part C: Insert Sample Data



```sql
-- Use to check inserted data
SELECT * FROM Person;
SELECT * FROM Departments;
SELECT * FROM Staff;
SELECT * FROM Doctors;
SELECT * FROM Nurses;
SELECT * FROM Patients;
SELECT * FROM Admissions;
SELECT * FROM Appointments;
```

100 % No issues found Ln: 207, Ch: 1 (262 chars, 11 lines) SPC CRLF UTF-8

Results Messages

	PersonID	FName	LName	Address	DOB	Gender
1	1	Amal	Perera	12 Galle Road, Colombo	1985-03-14	M
2	2	Nimali	Fernando	45 Kandy Road, Kandy	1990-07-22	F
3	3	Sunil	Jayasur...	78 Negombo Road, N...	1978-11-05	M
4	4	Kumari	Silva	23 Galle Face, Colombo	1982-01-30	F

	DeptID	DeptName
1	1	Cardiology
2	2	Neurology
3	3	Pediatrics

	StaffID	HireDate
1	1	2015-06-01
2	2	2018-03-15
3	3	2010-09-10
4	4	2019-01-20
5	5	2021-07-05
6	6	2016-11-11

	DocID	Specialization	DeptID
1	1	Cardiologist	1
2	2	Neurologist	2
3	3	Pediatrician	3

	NurseID	AddID
1	6	1
2	7	2
3	8	3

	PatientID	BloodGroup
1	11	O+
2	12	A-
3	13	B+

	AddID	BedNo	AddDate	DisDate	PatientID	NurseID
1	1	B101	2026-06-01	2026-06-05	11	6
2	2	B102	2026-06-10	NULL	12	7
3	3	B103	2026-06-15	2026-06-18	13	8
4	4	B104	2026-06-20	NULL	14	9

Query executed successfully. GIGABART (17.0 RTM) GIGABART\mathe (59) master 00:00:00 Row: 1, Col: 1 60 rows

Part D – SQL Queries & Outputs

```sql
-- Simple SELECT
-- List all patients with blood group O+
SELECT PatientID, BloodGroup
FROM Patients
WHERE BloodGroup = 'O+';


-- JOIN
-- List each doctor's full name, specialization, and department name
-- (joins Doctors -> Staff -> Person for the name, and Doctors -> Departments)
SELECT
    p.FName + ' ' + p.LName AS DoctorName,
    d.Specialization,
    dept.DeptName
FROM Doctors d
JOIN Staff s   ON d.DocID = s.StaffID
JOIN Person p  ON s.StaffID = p.PersonID
JOIN Departments dept ON d.DeptID = dept.DeptID;


-- Aggregation
-- Count the total number of doctors and total number of nurses
SELECT
    (SELECT COUNT(*) FROM Doctors) AS TotalDoctors,
    (SELECT COUNT(*) FROM Nurses)  AS TotalNurses;


-- GROUP BY / HAVING
-- Show departments that have more than 1 doctor assigned
SELECT
    dept.DeptName,
    COUNT(d.DocID) AS NumberOfDoctors
FROM Departments dept
JOIN Doctors d ON dept.DeptID = d.DeptID
GROUP BY dept.DeptName
HAVING COUNT(d.DocID) > 1;


-- Subquery
-- List patients who currently have an active admission (no discharge date yet)
SELECT
    p.FName + ' ' + p.LName AS PatientName,
    pat.BloodGroup
FROM Patients pat
JOIN Person p ON pat.PatientID = p.PersonID
WHERE pat.PatientID IN (
    SELECT PatientID
    FROM Admissions
    WHERE DisDate IS NULL
);
```

Results | Messages

	PatientID	BloodGroup
1	11	O+

	DoctorName	Specialization	DeptName
1	Amal Perera	Cardiologist	Cardiology
2	Nimali Fernando	Neurologist	Neurology
3	Sunil Jayasuriya	Pediatrician	Pediatrics
4	Kumari Silva	Orthopedic ...	Orthope...
5	Ruwan Bandara	General Ph...	General ...

	TotalDoctors	TotalNurses
1	5	5

	DeptName	NumberOfDoctors

	PatientName	BloodGroup
1	Chathura Herath	A-
2	Nuwan Dissanayake	AB+

Part E – Stored Function/Procedure

```sql
-- Part E: Stored Function/Procedure
-- Admits a patient: creates a new Admissions record and links the
-- assigned nurse's AddID to point at this new admission.
GO
CREATE PROCEDURE usp_AdmitPatient
    @PatientID  INT,
    @NurseID    INT,
    @BedNo      VARCHAR(10),
    @AddDate    DATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Admissions (BedNo, AddDate, DisDate, PatientID, NurseID)
    VALUES (@BedNo, @AddDate, NULL, @PatientID, @NurseID);

    DECLARE @NewAddID INT = SCOPE_IDENTITY();

    UPDATE Nurses
    SET AddID = @NewAddID
    WHERE NurseID = @NurseID;

    SELECT @NewAddID AS NewAdmissionID;
END

GO

-- Execution of created procedure
EXEC usp_AdmitPatient
    @PatientID = 13,
    @NurseID   = 8,
    @BedNo     = 'B106',
    @AddDate   = '2026-07-01';

-- Verify the result
SELECT * FROM Admissions WHERE PatientID = 13;
SELECT * FROM Nurses WHERE NurseID = 8;
```

Results | Messages

	NewAdmissionID
1	6

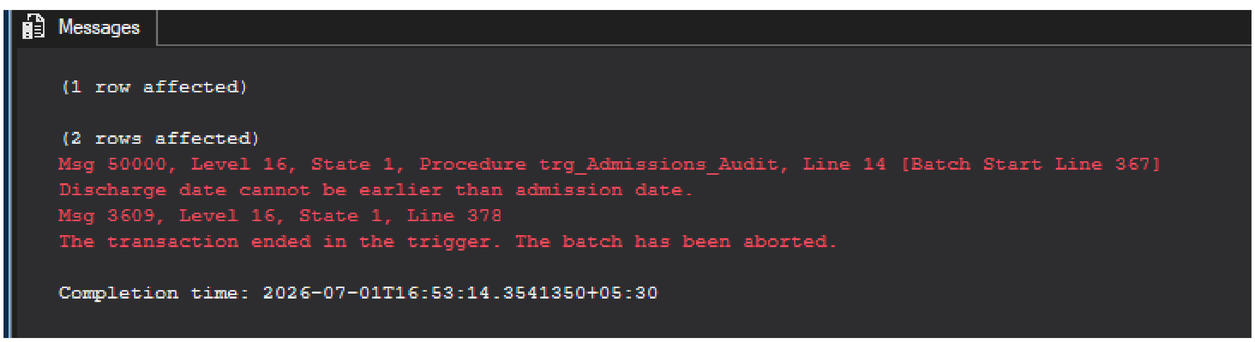
	AddID	BedNo	AddDate	DisDate	PatientID	NurseID
1	3	B103	2026-06-15	2026-06-18	13	8
2	6	B106	2026-07-01	NULL	13	8

	NurseID	AddID
1	8	6

Part F – Trigger

```
-- Part F: Triggers
-- Audit table to log every insert/update on Admissions
CREATE TABLE AdmissionsAudit
(
    AuditID     INT IDENTITY(1,1) NOT NULL,
    AddID       INT               NOT NULL,
    ActionType  VARCHAR(10)       NOT NULL,
    ActionDate  DATETIME          NOT NULL DEFAULT GETDATE(),
    Details     VARCHAR(255)      NULL,
    CONSTRAINT PK_AdmissionsAudit PRIMARY KEY (AuditID)
);

GO

-- Trigger: validates DisDate is not earlier than AddDate,
-- and logs every insert/update on Admissions into AdmissionsAudit
CREATE TRIGGER trg_Admissions_Audit
ON Admissions
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Validation: reject if DisDate is earlier than AddDate
    IF EXISTS (SELECT 1 FROM inserted WHERE DisDate IS NOT NULL AND DisDate < AddDate)
    BEGIN
        RAISERROR('Discharge date cannot be earlier than admission date.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Audit logging: distinguishes INSERT vs UPDATE by checking the deleted table
    INSERT INTO AdmissionsAudit (AddID, ActionType, Details)
    SELECT
        i.AddID,
        CASE WHEN EXISTS (SELECT 1 FROM deleted d WHERE d.AddID = i.AddID)
             THEN 'UPDATE' ELSE 'INSERT' END,
        'BedNo=' + ISNULL(i.BedNo, 'NULL')
```

```sql
            + ', PatientID=' + CAST(i.PatientID AS VARCHAR)
            + ', AddDate=' + ISNULL(CONVERT(VARCHAR, i.AddDate, 120), 'NULL')
            + ', DisDate=' + ISNULL(CONVERT(VARCHAR, i.DisDate, 120), 'NULL')
    FROM inserted i;
END

GO

-- Demonstration 1: valid INSERT -> should succeed and log an INSERT entry
INSERT INTO Admissions (BedNo, AddDate, DisDate, PatientID, NurseID)
VALUES ('B107', '2026-06-28', NULL, 11, 6);

-- Demonstration 2: valid UPDATE (setting a discharge date) -> should succeed and log
an UPDATE entry
UPDATE Admissions
SET DisDate = '2026-07-01'
WHERE BedNo = 'B107' AND PatientID = 11;

-- Demonstration 3: invalid UPDATE (DisDate earlier than AddDate) -> should fail and
roll back
UPDATE Admissions
SET DisDate = '2026-01-01'
WHERE BedNo = 'B107' AND PatientID = 11;

-- Check the results
SELECT * FROM AdmissionsAudit;
SELECT * FROM Admissions WHERE BedNo = 'B107';
```



```
Messages

(1 row affected)

(2 rows affected)
Msg 50000, Level 16, State 1, Procedure trg_Admissions_Audit, Line 14 [Batch Start Line 367]
Discharge date cannot be earlier than admission date.
Msg 3609, Level 16, State 1, Line 378
The transaction ended in the trigger. The batch has been aborted.

Completion time: 2026-07-01T16:53:14.3541350+05:30
```

The error shown means the trigger is actually working since the code in Demonstration 3 shouldn't run and has to rollback

When Demonstration 1 and 2 is run it works without errors

Results | Messages

	AuditID	AddID	ActionType	ActionDate	Details
1	1	7	INSERT	2026-07-01 16:49:36.880	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
2	2	7	UPDATE	2026-07-01 16:49:36.883	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
3	3	8	INSERT	2026-07-01 16:53:14.340	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
4	4	8	UPDATE	2026-07-01 16:53:14.347	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
5	5	7	UPDATE	2026-07-01 16:53:14.347	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
6	6	9	INSERT	2026-07-01 16:55:56.373	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
7	7	9	UPDATE	2026-07-01 16:55:56.380	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
8	8	8	UPDATE	2026-07-01 16:55:56.380	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...
9	9	7	UPDATE	2026-07-01 16:55:56.380	BedNo=B107, PatientID=11, AddDate=2026-06-28, Dis...

	AddID	BedNo	AddDate	DisDate	PatientID	NurseID
1	7	B107	2026-06-28	2026-07-01	11	6
2	8	B107	2026-06-28	2026-07-01	11	6
3	9	B107	2026-06-28	2026-07-01	11	6